

PROJECT PROPOSAL

DankangNote

협업 SaaS 프로젝트 기획서

문서 · 채팅 · 보드를 하나의 워크스페이스에서 — 노션+슬랙+지라형 올인원 협업 도구

문서 버전	v1.0
작성일	2026년 7월 23일
작성자	DanKang (ekffy99@gmail.com)
저장소	github.com/DankangE/DankangNote
이슈 트래커	ekffy99.atlassian.net — 프로젝트 KAN

1. 프로젝트 개요

DankangNote는 팀의 문서(노트), 실시간 채팅, 칸반 보드를 하나의 워크스페이스에서 제공하는 **멀티테넌트 협업 SaaS**다. Notion(문서) · Slack(채팅) · Jira(보드)를 참고 모델로 하며, 실무 수준의 SaaS 아키텍처를 처음부터 끝까지 직접 설계·구현하는 **학습 및 포트폴리오 프로젝트**다.

단순 클론 코딩이 아니라, 실제 SaaS 제품 개발에서 마주치는 문제들 — 멀티테넌시 데이터 격리, 인증·조직 관리, 외부 서비스(Clerk) 웹훅 동기화, 에러 계약 설계 — 을 정면으로 다루는 것을 목표로 한다. 모든 작업은 Jira 티켓 기반으로 진행하고, PR마다 멀티 에이전트 자체 코드리뷰를 거쳐 실무형 개발 프로세스 자체도 포트폴리오의 일부로 만든다.

2. 목표와 범위

제품 목표

- 워크스페이스 단위로 완전히 격리된 팀 협업 공간 제공
- 노트 작성 → 리치 문서 편집 → 실시간 채팅 → 칸반 보드로 기능 확장
- 초대·역할 기반의 멤버 관리

학습 목표

- Next.js 16 App Router + Server Actions 실전 활용
- 멀티테넌시 설계 — orgId 스코프 강제, 교차 테넌트 차단
- Clerk 기반 인증·조직 + 웹훅 DB 동기화 패턴
- Jira 티켓 → 브랜치 → PR → 리뷰의 실무 워크플로우 체화

■ 범위 밖 (Non-Goals)

- 결제·과금, 요금제 관리
- 대규모 트래픽 대응(수평 확장, 캐시 계층) — 설계 시 고려는 하되 구현하지 않음
- 네이티브 모바일 앱, 오프라인 동기화

■ 타겟 사용자

스터디 그룹, 사이드 프로젝트 팀 등 **2~10인 소규모 팀**. 문서·대화할 일이 여러 도구에 흩어져 있는 팀이 도구 하나로 협업의 최소 단위를 해결하는 시나리오를 가정한다.

3. 핵심 기능

- F1 워크스페이스 & 멤버 관리

진행 중

Clerk Organization을 워크스페이스로 매핑. 로그인 → 조직 선택/생성 → 해당 조직 데이터만 접근. 웹훅으로 사용자·조직·멤버십을 DB에 미러링해 작성자 귀속, 멤버 목록, 역할 권한의 토대를 마련한다. 초대·역할 기반 권한 강제는 후속 단계.
- F2 노트 (문서)

CRUD 완료

워크스페이스 스코프의 노트 생성·조회·수정·삭제와 목록/편집 UI 구현 완료. 이후 Tiptap 기반 리치 텍스트 에디터로 고도화하고, 낙관적 업데이트(React 19 useOptimistic)로 UX를 다듬는다.
- F3 실시간 채팅

예정

Pusher 기반 워크스페이스 채널 채팅. 멤버 간 실시간 메시지 송수신, 입장/퇴장 표시.
- F4 칸반 보드

예정

dnd-kit 기반 드래그 앤 드롭 보드. 컬럼·카드 관리로 팀의 할 일을 시각화한다.

4. 기술 스택

영역	선택	비고
프레임워크	Next.js 16 (App Router) + TypeScript	strict 모드, any 금지
UI / 스타일링	Astryx 디자인 시스템 + StyleX	Tailwind·shadcn 미사용, StyleX 단독
DB / ORM	PostgreSQL 17 + Prisma 7	driver adapter(@prisma/adapter-pg), 로컬은 docker-compose
인증 · 멀티테넌시	Clerk	Organization = 워크스페이스, 웹훅 DB 동기화
상태 관리	TanStack Query / Zustand	서버 상태 / 클라이언트 상태 (도입 예정)
도메인 라이브러리	Tiptap · Pusher · dnd-kit	문서 편집 · 실시간 채팅 · 보드 DnD (도입 예정)
검증	zod	모든 외부 입력은 서버에서 검증 후 사용
개발 환경	WSL2 + pnpm 10 + Node 22 (fnm)	React 19, Turbopack

5. 아키텍처와 설계 원칙

폴더 구조 — 기능별 수직 슬라이스 + 서버 전용 레이어

```
src/
  app/                                # 라우팅 (App Router)
  features/<domain>/                  # 기능별 수직 슬라이스
    api/actions.ts                   # Server Action (진입부 인증 + zod 검증)
    api/queries.ts                   # 서버 컴포넌트용 조회
    components/                      # 해당 기능 UI
  server/                             # 서버 전용 레이어 (server-only 강제)
    db.ts                            # Prisma 싱글톤
    auth.ts                          # requireOrgId() - Clerk auth() 연동
    services/                        # 도메인 서비스 (모든 쿼리 orgId 스코프)
```

설계 원칙

원칙	내용
멀티테넌시 격리 (최우선)	테넌트 데이터 쿼리는 반드시 <code>where: { orgId }</code> 로 스코프. 스코프 없는 조회는 버그로 간주. update/delete는 <code>updateMany / deleteMany</code> 로 교차 테넌트 접근을 쿼리 수준에서 원천 차단.
변이는 Server Action 우선	Route Handler는 웹훅 등 외부 노출이 필요한 경우에만. 모든 액션은 진입부에서 공용 헬퍼로 인증.orgId 확인.
에러 계약 (ActionResult)	액션은 <code>{ ok: true } { ok: false, error }</code> 판별 유니언만 반환. DB 예외가 클라이언트로 새지 않도록 guarded 래퍼로 감싼다.
입력 검증	폼·파라미터·웹훅 페이로드 등 모든 외부 입력은 서버에서 zod 검증 후 사용.
Clerk 미러링	사용자/조직/멤버십은 웹훅(<code>user.* , organization.* , organizationMembership.*</code>)으로만 DB에 동기화. 멍등 upsert + 단일 트랜잭션으로 웹훅 배달 순서와 무관하게 FK 안전 보장.

6. 데이터 모델

Note	User (Clerk 미러)	Organization (Clerk 미러)	Membership (Clerk 미러)
id cuid PK orgId 테넌트 스코프 title / content createdAt / updatedAt	id user_... PK email / firstName... ↔ memberships	id org_... PK name / slug ↔ memberships	id orgmem_... PK orgId → Organization userId → User role org:admin...

- Note.orgId 는 의도적으로 FK가 아닌 문자열 — 노트 생성이 웹훅 동기화 순서에 묶이지 않게 하기 위한 부트스트랩 경합 회피. FK 소급은 후속 티켓.
- 미러 3종(User/Organization/Membership)은 웹훅으로만 기록되며, Membership 은 (orgId, userId) 유니크 제약으로 중복 소속을 차단한다.

7. 개발 프로세스 · 품질 관리

- 티켓 기반 개발 — 모든 작업은 Jira(KAN) 티켓 생성 → feat/KAN-N-slug 브랜치 → 커밋·PR 제목에 이슈 키 포함.
- 멀티 에이전트 자체 코드리뷰 — PR마다 관점별 finder 에이전트가 후보를 발굴하고, 발견별 독립 검증을 거쳐 확정된 것만 인라인 코멘트로 게시. 확정 이슈는 후속 티켓으로 분리해 추적한다. (예: PR #1 — 후보 14건 → 8건 확정, KAN-5/6/7로 분리)
- 코딩 룰의 문서화 — .claude/rules/ (general-architecture-backend-frontend)에 규칙을 명문화해 세션이 바뀌어도 일관성 유지.
- 런타임 검증 — 타입·린트 통과에 그치지 않고 dev 서버에 HTTP를 직접 호출해 정상/오류/교차 테넌트 시나리오를 실측 검증.

8. 로드맵

단계	내용	관련 이슈 / PR	상태
Phase 0 기반	Next.js 16 초기화, Astryx 디자인 시스템 셋업. 렌더 검증, 개발 룰 정착	—	완료
Phase 1 노트 도메인	노트 CRUD API(멀티테넌시 스코프) → 빌드 안정화 → 에러 계약 강화 → 노트 목록/편집 UI	KAN-4~6, KAN-8 PR #1~#4 머지	완료
Phase 2 인증. 멀티테넌시	Clerk 인증 뼈대(Organization=워크스페이스, requireOrgId 실연동) 완료 → 웹훅 기반 User/Org/Membership DB 동기화 진행 중	KAN-10 (PR #5 머지) KAN-11 (현재 브랜치)	진행 중
Phase 3 문서 고도화	TanStack Query-Zustand 도입, 낙관적 업데이트 (KAN-9), Tiptap 리치 에디터	KAN-9 외	예정
Phase 4 협업 확장	Pusher 실시간 채팅, dnd-kit 칸반 보드, 멤버 초대·역할 기반 권한 강제, Note FK 소급	—	예정

백로그: KAN-7(리뷰 cleanup 일괄 처리), KAN-9(노트 UI 낙관적 업데이트) — Phase 3에서 소화 예정.

9. 리스크와 대응

리스크	대응
최신 스택(Next 16, Prisma 7, React 19)의 breaking change — 학습 데이터와 실제 API 불일치	추측 금지. <code>node_modules</code> 내 공식 문서·CLI로 확인 후 진행하는 것을 룰로 강제
1인 개발 — 리뷰어 부재로 인한 품질 사각지대	PR마다 멀티 에이전트 자체 리뷰(발굴→독립 검증→확정)를 필수 프로세스로 운영
Clerk 웹훅 로컬 테스트 제약 (localhost 직접 수신 불가)	서명 검증·역등성 단위 검증 우선, e2e는 터널(ngrok/cloudflared)로 별도 수행
웹훅 배달 순서/유실로 인한 미리 데이터 불일치	역등 upsert + 멤버십 이벤트에서 <code>org-user-membership</code> 을 단일 트랜잭션 upsert
WSL·Node 버전 파편화 (시스템 v20 vs 프로젝트 v22)	<code>.node-version</code> 고정 + <code>fnm</code> 스코프 실행으로 재현 가능한 환경 유지